

Advanced Data Structures for Olympiad Programmers

1	Introduction	3
1.1	Notation	3
1.2	Graph Theory	3
2	Fenwick Trees	4
2.1	lowbit	4
2.2	Structure	4
2.3	Point Update	5
2.4	Prefix Sum Query	5
2.5	Point Query	6
2.6	lowerbound; Binary Lifting on Fenwick	6
2.7	Subarray Queries	7
3	Segment Tree	7
3.1	Structure	7
3.2	Decomposing a range into segment tree nodes.	8
3.3	Point Update Range Query	9
3.4	Basic Algebraic Structures	10
3.5	Generalization of Point-Update-Range-Query Segment Trees	11
3.6	Range Update Point Query	12
3.7	Generalization of Range-Update-Point-Query	13
3.8	Pool-allocated Segment Tree	14
3.9	Sparse Segment Tree	15
3.10	Walking on Segment Tree	16
3.11	Range Update Range Query: Pushdown-Free	17
3.12	Range Update Range Query: Lazy Propagation.	17
3.13	Generalization of Lazy Propagation	19
4	Linear Algebra	19
5	Tree Flattening	21
6	DSU on Tree	21
7	Dynacon	21
8	References	22

1 Introduction

1.1 Notation

First, I shall lay out the terms which are used later onto this document. On a cartesian plane, point O means the origin, the main horizontal axis is the line $y = 0$ or the OX axis, and the main vertical axis is the line $x = 0$ or the OY axis. Given a point $P = (a, b)$ on the plane, we say that the quantity b is the *ordinate* of point P .

On a cartesian space, there are axes OX, OY, OZ . The plane containing OX and OY is the OXY plane. The OXZ and OYZ planes are defined analogously. A point $P = (a, b, c)$ on the space has *abscissa* a , *ordinate* b , *applicate* c . But on a four-dimensional space, there is no such standard name for the fourth coordinate, hence we call it that, the *fourth coordinate*.

When a set of N objects with a total order (e.g., the integer set $\{4, 29, 482, -4892, 99999\}$) is given, we can provide a function from each member of the set into unique identification integer ranging from 1 to N . This process we call *discretization* or *coordinate compression*.

All logarithms are in base two, unless stated otherwise.

1.2 Graph Theory

A graph $G = (V, E)$ consists of a set of vertices V and a set of edges E . A graph is also called an undirected simple graph. We typically denote the *order* (count of vertices) by $|V| = N$ and the *size* (count of edges) by $|E| = M$.

Two vertices are *adjacent* in a graph iff there exists an edge connecting them. A vertex and an edge are *incident* iff the vertex is an endpoint of that edge. A *walk* is a list of vertices where each two consecutive vertices are adjacent. A *trail* is a walk where no edge is repeated. A *path* is a trail where no vertex is repeated. A *cycle* is a walk where no vertex is repeated except that the first equals the last.

A *tree* is a graph satisfying any of the following statements: (i) is a minimally connected graph. (ii) its size is one less than its order. (iii) is a maximally acyclic graph. These three statements are equivalent. Removing any edge cuts it into two. Adding any edge makes a cycle. Given any two different vertices u, v in a tree, there is only one path between the vertices; we call this the $u - v$ path.

A rooted tree is a tree where one vertex, r , is chosen as root. In a rooted tree, u is an ancestor of v iff the $r - v$ path contains u . When u is an ancestor of v , the v is a descendant of the u ; note that u and v may be the same. The depth of vertex u , $depth(u)$ is the number of edges in $r - u$ path. Vertex p is a parent of u iff they are adjacent and p is one ancestor of u . The *subtree* of u is the subgraph of the original tree, where only descendants of u exist. The *lowest common ancestor* of u and v , $LCA(u, v)$ is the shared ancestor of both vertex with maximum depth.

On a rooted tree, we may perform *tree flattening*. The process is running a depth-first-search starting at root r , generating the *discovery time* and *exit time* of

each vertex (integer timestamps); the former is denoted $dfn[u]$ and is one plus the number of vertices visited before it. The latter is denoted $tout[u]$ and is the discovery time added by one less than the order of subtree of u . The discovery time of all vertices together, is a permutation of $1 \dots N$. Its inverse permutation, we call $nfd[]$; it satisfies $nfd[dfn[u]] = u$. If one take any vertex u and consider the set generated by $\{nfd[x] : x \in [dfn[u], tout[u]]\}$, that set is precisely the vertices in the subtree of u .

2 Fenwick Trees

A *Fenwick Tree*, also called *Binary Indexed Tree (BIT)*, is a data structure providing efficient method for maintaining an fixed-sized list supporting *point-add* and *prefix-sum* queries. Both operations run in $O(\log N)$ time, where N is the size of the array. It consumes $O(N)$ memory.

It solves the problem below in $O(\log N)$ time per query.

Example 1. Maintain an array of integers $A[1..N]$ of length N supporting these two types of operation: $update(i, val)$ adds val to $A[i]$. $query(i)$ asks for the sum $A[1] + A[2] + \dots + A[i]$.

2.1 lowbit

The key operation is $lowbit(i)$, which returns the value of the least significant set bit of i . For example, $lowbit(12) = lowbit(1100_2) = 100_2 = 4$.

To implement $lowbit$ in constant operation, we should first understand Two's Complement. In most if not all computers to-day, negative integers are internally stored with Two's complements. Let us consider a hypothetical four-bit signed integer types. 5 would be represented as 0101. What would -5 be?

Since the integer type holds four bits, any overflow is truncated. That is, if we add 0101 to 1011, resulting in 10000, that high bit cannot be stored, hence the result is 0000. And that is the principle of Two's Complement; any number added to its negative must be zero. Then, one can see that the binary representation of $-x$ is obtained by flipping all the bits in x then adding one to it. Now, the $lowbit$ operation is obtained by: $lowbit(x) = x \& -x$.

2.2 Structure

A Fenwick Tree is an array $T[]$. It completely contains all the information of the array $A[]$, and storing the actual A is redundant. We maintain:

$$T_i = \sum_{j=i-lowbit(i)+1}^i A_j$$

This makes a hierarchical decomposition. Each index i is responsible for a range of size $lowbit(i)$.

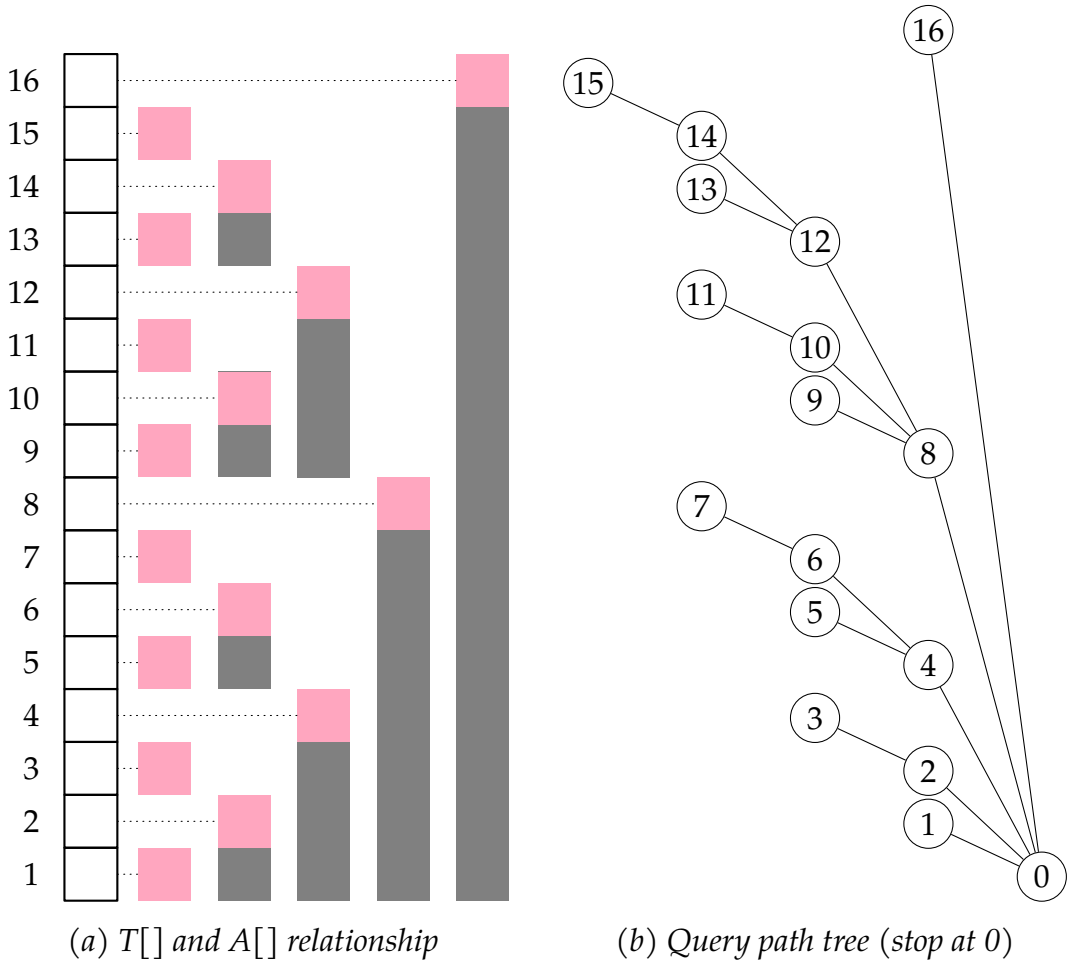


Figure: Fenwick tree for $n = 16$.

2.3 Point Update

To add value v to position $A[p]$:

```
void update(int p, int v) {
    for (; p <= n; p += lowbit(p))
        T[p] += v;
}
```

The function adds v to $T[j]$ for each j whose responsible range contains p .

Lemma 2. The update procedure modifies $O(\log N)$ indices.

Proof. Each iteration flips the least significant set bit. The number of set bits in N cannot exceed $\lceil \log_2 N \rceil$. $\square$

2.4 Prefix Sum Query

To compute $query(k) = \sum_{i=1}^k A[i]$:

```

int query(int p) {
    int res = 0;
    for (; p > 0; p -= p & -p)
        res += T[p];
    return res;
}

```

The algorithm climbs down the ‘staircase’ (see the figure for query path tree), fetching the sum from the indices along the way. The ranges that belong to those indices are disjoint. Consider, $query(13) = query(1011_2)$; it visits $T[13], T[12], T[8]$. The ranges $[13, 13], [9, 12], [1, 8]$ are pairwise disjoint and together makes $[1, 13]$.

Lemma 3. The query procedure visits $O(\log N)$ indices.

Proof. Each iteration flips the least significant set bit. The number of set bits in N cannot exceed $\lceil \log_2 N \rceil$. $\square$

2.5 Point Query

Of course, one can obtain $\sum_{i=a}^b A[i]$ by $query(b) - query(a-1)$. And $A[i] = query(i) - query(i-1)$, but it is wasteful. The special case of point query can be optimized as follows.

```

int getpoint(i) {
    int res = T[i];
    int z = i - (i & -i);
    for (i--; i != z; i -= i & -i)
        res -= T[i];
    return res;
}

```

The code first take the sum in the range of $(i - lowbit(i), i]$, then subtract the unwanted contributions.

2.6 lowerbound; Binary Lifting on Fenwick

It is often desirable to find the first index p such that $\sum_{i=1}^p A_i \geq W$. Of course, performing a binary search together with querying the tree yields $O(N \log^2 N)$. But there exists a $O(N \log N)$ solution:

```

int lower_bound_fw(int W) {
    int p = 0;
    for (int k = 1 << log2[N]; k; k /= 2)
        if (p + k <= N && T[p + k] < W)
            W -= T[p += k];
    return p + 1;
}

```

Example 4. Inversion Count

Given an array $A[1..N]$, count the number of pairs (i, j) such that $i < j$ and $A[i] > A[j]$.

Solution

First, discretize the array so that elements are in range $1 \dots N$. Process elements from right to left. For each position i , query the BIT for the number of elements smaller than $A[i]$ seen so far. Then, add $A[i]$ to the BIT.

```
long long count_inversions(int A[], int N) {
    Discretize(A, N);
    memset(T, 0, sizeof T);
    long long ans = 0;
    for (int i = N; i >= 1; i--) {
        ans += query(A[i] - 1);
        update(A[i], 1);
    }
    return ans;
}
```

Time complexity: $O(N \log N)$.

2.7 Subarray Queries

Fenwick tree also supports prefix max and changemax operations. The implementation is analogous to the sum fenwick tree.

It may surprise you, Fenwick trees also support general subarray queries, not just prefix queries, while retaining fast $O(N \log N)$ time. However, this function is trivialized by *segment trees*, which we discuss next. And the level of details of this Fenwick subarray trick is too troublesome. If you are interested, see [1].

3 Segment Tree

The *Segment Tree* is the usefulest data structure that you shall master. In its simplest form, it handles point modification and range query for any associative operations, not requiring inversibility

3.1 Structure

See the figure below, where $A[]$ is the array of data we wish to manipulate, and $T[]$ is the array used for the segment tree; the number in each box is the corresponding index.

The root of segment tree—that is node 1—covers the entirety of $A[]$. A non-leaf node of the segment tree has leftchild and rightchild, being equal in size. In each node is stored the aggregate (in the information type we wish to know) of its range.

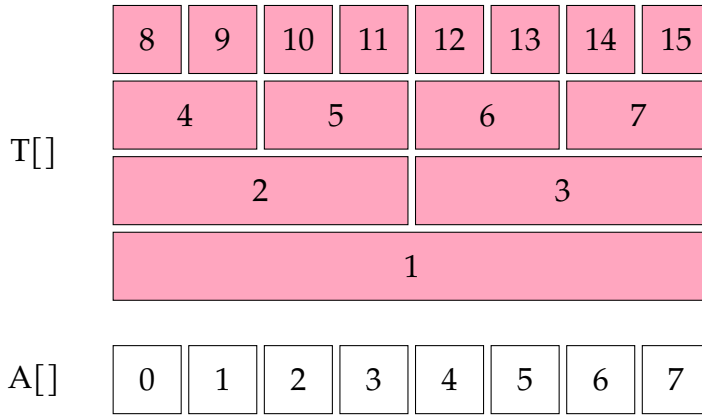


Figure 3.1 [Segment tree for $N = 8$]

On the standard numbering for segment tree (as shown), the root of the segment tree is stored at index 1. The label for $lc(u)$ is $2u$ and for $rc(u)$, $2u + 1$. It can be proven that in such tree, the maximum index used will not exceed $4N$. The proof is omitted.

Each node of the tree has a range for which it takes responsibility (that is, stores the appropriate aggregate). For example, node 5 in the figure is responsible for $[2, 3]$.

```
#define lc(u) (2*(u))
#define rc(u) (2*(u) + 1)
```

3.2 Decomposing a range into segment tree nodes.

When a range (subarray) $[l, r]$ is given, there exists a set of nodes whose ranges are disjoint and their union make up precisely $[l, r]$. The process of finding that set is shown in the code below. Henceforth, this set associated with range $[l, r]$ is denoted by $cover(l, r)$.

```
void getcover(int v, int tl, int tr, int l, int r) {
    if (tr < l || r < tl)
        return;
    if (l <= tl && tr <= r) {
        /* current node is in the set */
    } else {
        getcover(lc(v), tl, (tl + tr) / 2, l, r);
        getcover(rc(v), (tl + tr) / 2 + 1, tr, l, r);
    }
}
```

Lemma 5. The size of $cover(l, r)$ is $O \log N$.

Proof. The number of visited node is the same order as the number of *partial cells* visited. Define a partial cell as one that falls into the last else branch, i.e.,

partially overlapping. Such cell's responsible range either contains l or contains r . Thus, it must be an ancestor of l or an ancestor of r (or both). Since the segment tree's height is $O(\log N)$, that number of ancestors is $O(\log N)$. $\square$

3.3 Point Update Range Query

To be concrete, let us consider a max segment tree which solves the following problem.

Example 6. Maintain an array $A[1..N]$, initially zeroes, supporting operations as follow: (i) assign $A[p] = v$. (ii) fetch $\max_{i=l}^r A[i]$.

First, since *assign* operation is required, we need not create a build operation—we can just perform *assign*($i, A[i]$) for all i .

Assign

Remember the invariant of this segment tree; for the segment tree node u whose responsible range is $A[l_u] \dots A[r_u]$, the value $T[u]$ must equal $\max_{i=l_u}^{r_u} A[i]$.

Thus, to perform *assign*(p, v), we must first update the leaf node which is responsible for p (let that node be l). That is, set $T[l] = v$. Then, notice that after doing so the invariant is broken for the ancestor nodes of l (all the other nodes covering p), because they have not received the new information. To fix this, we climb up the ancestors—if p is 4 then l is 8, we traverse $12 - 6 - 3 - 1$ —and for each ancestor a update $T[a]$ to be $\max(T[lc(a)], T[rc(a)])$. This works because if the values in the children node are correct, the aggregate of those values is the correct value for the parent.

First, it is most important to clarify the parameter naming. v is the integer label that is index to the array $T[]$. tl and tr together make a closed interval $[tl, tr]$ that is the range which v is responsible for—we call this tree range. The last two parameters are information of the update: we wish to assign $A[p] = v$.

```
void assign(int v, int tl, int tr, int p, int val) {
    if (tl == tr) {
        T[v] = val;
        return;
    }
    if (p <= (tl + tr) / 2)
        assign(lc(v), tl, (tl + tr) / 2, p, val);
    else
        assign(rc(v), (tl + tr) / 2 + 1, tr, p, val);
    T[v] = max(T[lc(v)], T[rc(v)]);
}
```

The procedure visits no more nodes than the height of the segment tree— $O(\log N)$.

The logic of updating a non-leaf node's value by those of its children is the same for most type of segment trees—not just max segment tree. Hence, we may abstract it as:

```

void pushup(int v, int tl, int tr) {
    T[v] = max(T[lc(v)], T[rc(v)]);
}

```

Then replace the last line in `assign` with `pushup(v)`. It may seem pointless for `pushup` to know the tree range. However we see later (in lazy propagation) that it can be crucial.

Query

```

#define LEAST -1e9
int query(int v, int tl, int tr, int l, int r) {
    if (tr < l || r < tl) return LEAST;
    /* visit v */
    if (l <= tl && tr <= r) return T[v];
    return max(query(lc(v), tl, (tl + tr) / 2, l, r),
               query(rc(v), (tl + tr) / 2 + 1, l, r));
}

```

There are three cases; (i) the two ranges don't overlap: in this case it is useless to do further work on that part of segment tree. (ii) the tree range is wholly covered by query range: in this case all values which v is responsible for shall be considered, and all those values are aggregated in $T[v]$, hence we can simply return it. (iii) the two ranges partially overlap: here, we descend down the tree on both children, by doing so we have not lost any information.

3.4 Basic Algebraic Structures

To move from simple segment trees to more complex ones, it is important to understand what they can and cannot do, and how precisely they work.

A binary operation $\circ$ on set S is a function $\circ : S^2 \rightarrow S$. It is commutative iff $a \circ b$ equals $b \circ a$. It is associative if and only if $(a \circ b) \circ c$ equals $a \circ (b \circ c)$.

A set of objects S together with $\circ$, a binary operation on it, is called a *magma*, and is denoted $(S, \circ)$.

Listed are some examples of magma.

- $Z_2 = \{0, 1\}$ with $\circ = \{((0, 1), 0), ((0, 0), 1), ((1, 0), 0), ((1, 1), 1)\}$
- $A = \{1, 2, 3, 4\}$ with $a \circ b$ returning one iff either a or b is one and returning zero otherwise.

Exercise 7. Find examples of three other magma.

A magma $(S, \circ)$ whose $\circ$ is *associative* is called a *semigroup*. For example, the set $Z_2 = \{0, 1\}$ with $\circ$ being the XOR operation, is a semigroup.

Exercise 8. Prove that $(N, \min)$ is a semigroup.

Exercise 9. Find examples of three other semigroups.

Exercise 10. Let S be the set of all strings over alphabet a, b , and let $\circ$ be concatenation. Prove that $(S, \circ)$ is a semigroup.

A semigroup $(S, \circ)$ that also contains an identity element e to which applied any element returns that element, is called a *monoid*, and is denoted $(S, \circ, e)$. For example, the set of real numbers, together with the standard addition operation, has zero as its identity element, and thus is a monoid.

Exercise 11. Find examples of three other monoids.

3.5 Generalization of Point-Update-Range-Query Segment Trees

To move from simple segment trees to more complex ones, it is important to understand what they can and cannot do, and how precisely they work.

Let S be the set of possible values that the segment tree may store in each of its nodes. And let $\circ$ be the merge operation on two elements from S ; notice that $\circ$ must be associative. This is because a non-leaf node merge values from its two child; if both of those children are non-leaf and $\circ$ is not associative, the value for that node will be wrong. The standard point-update-range-query segment tree works if the structure forms a monoid $(S, \circ, e)$.

In fact, a segment tree can handle a semigroup, but the implementation is messy. Thus, it is preferable to, whenever you need to build a segment tree handling a semigroup, turn that semigroup into a monoid. That can be done by adding the identity element e . That is, to turn the semigroup $S_0, \circ_0$ into a monoid $S, \circ$, let $S = S_0 \cup e$ where e is an arbitrary object not in S_0 , and let $\circ = \circ_0 \cup \bigcup_{x \in S} \{((x, e), x), ((e, x), x)\}$.

Exercise 12. Extend the semigroup $(\mathbb{Z}, \max)$ onto a monoid.

Exercise 13. Extend the semigroup $(\mathbb{N}^+, \times)$ onto a monoid.

Exercise 14. Prove that the identity of a monoid must be unique.

Consider this implementation of segment tree for general point-update-range-query situations.

```
#define S /*type you want here*/>
#define MAXN 1000
#define mid ((tl + tr) / 2)

/* (S,merge) need not be commutative, but must be associative */
S merge(S l, S r) { /* any kind of associative merge */ }

S e = /* identity element here */

S T[4 * MAXN];
```

```

void pushup(int v, int tl, int tr) {
    T[v] = merge(T[lc(v)],
        T[rc(v)]);
}

void update(int v, int tl, int tr, int p, S val) {
    if (tl == tr) {
        T[v] = val;
        return;
    }
    if (p <= mid)
        update(lc(v), tl, mid, p, val);
    else
        update(rc(v), mid + 1, tr, p, val);
    pushup(v, tl, tr);
}

S query(int v, int tl, int tr, int l, int r) {
    if (tr < l || r < tl) return e;
    if (l <= tl && tr <= r) return T[v];
    return merge(query(lc(v), tl, mid, l, r),
        query(rc(v), mid + 1, tr, l, r));
}

void initsegtree(int n) {
    for (int i = 1; i < 4 * n; ++i)
        T[i] = e;
}

```

Exercise 15. Design a monoid for each of these:

* Range bitwise AND query. * Range greatest common divisor. * Range minimum query.

3.6 Range Update Point Query

The segment trees above handle point update range query, but often we wish to update on ranges and query on points.

Example 16. Maintain an array of integers $A[]$ of length N , initially zeroes, supporting these queries (i) *update*(l, r, x): do $A[i] = \max(A[i], x)$ for $i \in [l, r]$. or (ii) *query*(p) : print $A[p]$.

Solution. Maintain a segment tree $T[]$ where each node v stores $\text{tag}[v]$ —the best value from all previous *updates* that cover its range but not its parent’s

range. To handle $query(p)$, print maximum of the tag-values of all segment tree nodes covering p . For clarity, refer to the code below.

```

void update(int v, int tl, int tr, int l, int r, int x) {
    if (tr < l || r < tl)
        return;
    if (l <= tl && tr <= r) {
        if (tag[v] < x)
            tag[v] = x;
        return;
    }
    update(lc(v), tl, (tl + tr) / 2, l, r, x);
    update(rc(v), (tl + tr) / 2 + 1, tr, l, r, x);
}

int query(int v, int tl, int tr, int p) {
    int here = tag[v];
    if (tl == tr)
        return here;
    if (p <= mid)
        return max(here, query(lc(v), tl, mid, p));
    return max(here, query(rc(v), mid + 1, tr, p));
}

```

3.7 Generalization of Range-Update-Point-Query

We can summarize the algebraic structures needed for a range-update-point-query segment tree, as follows. Here we need the *update structure*, being a monoid: $(U, \diamond, id)$, where U is the set of all possible values x passed to $update(l, r, x)$. $\diamond$ is a merge operation closed on U . id is the identity element in U . Additionally is needed a function $\phi : U \rightarrow S$, which convert an update or sum of update to a value in the data type S of interest.

The array $tag[]$ of the segment tree is of type U . On an $update(l, r, x)$ operation, we, for the minimal set of nodes which together make up the range $[l, r]$, perform $tag[v] = tag[v] \diamond x$ for each node v in that set. On an $query(p)$ operation, we take the $\diamond$ -product of $tag[v]$ for all node v whose range contains p ; then pass that $\diamond$ -product through ϕ to obtain the final answer.

Example 17. For example 16, what is the monoid $(U, \diamond, id)$ and ϕ in use?

Solution. The monoid $(U, \diamond, id)$ is defined by U being set of integers, $\diamond$ being the max function, id being negative infinity, and ϕ being the identity function that does nothing, for the type of update-data and query-data need not be different—i.e., $\phi(x) = x$.

3.8 Pool-allocated Segment Tree

Of course, the heap-numbering (as presented in the previous chapter) is not the only implementation. So long as the tree have the balanced structure of a segment tree, you can manage the memory allocation in any manner.

Consider the following pool allocated segment tree (which is equivalent, in principle, to using pointers and `malloc`).

```
struct NODE {
    int lc, rc; /* children */
    S data;
} T[1 << 19]; /* the pool */
int ptr = 0;

int build(int tl, int tr) {
    int i = ptr++;
    T[i].data = e;
    T[i].lc = build(tl, mid);          T[i].rc = build(mid + 1, tr);
    return i;
}
```

This way you can make multiple segment trees, each of different sizes, effortlessly. The `\tt update` and `\tt query` procedures are implemented similarly.

```
\hisyntax{C}
\begin{code}
void update(int v, int tl, int tr, int p, S val) {
    if (tl == tr)    T[v].data = val;
    else {
        if (p <= mid) {
            update(T[v].lc, tl, mid, p, val),
            update(T[v].rc, mid + 1, tr, p, val);
        }
        pushup(v, tl, tr);
    }
}
```

This way you can make multiple segment trees, each of different sizes, effortlessly. The `update` and `query` procedures are implemented similarly.

```
void update(int v, int tl, int tr, int p, S val) {
    if (tl == tr)    T[v].data = val;
    else {
        if (p <= mid) {
            update(T[v].lc, tl, mid, p, val),
```

```

        update(T[v].rc, mid + 1, tr, p, val);
    }
    pushup(v, tl, tr);
}
}

```

Notice the children's indices are stored explicitly in the struct `NODE`. Granted, some memory are wasted this way. But this implementation allow us to make dynamic sized segment trees.

3.9 Sparse Segment Tree

Example 18. Maintain an array of one billion integers, initially filled with zeros, supporting point add update and range sum query; use $O(N \log 1000000000)$, forced online¹.

Solution.

Had we have a segment tree of billions of nodes, it would be trivial! Notice, that if the segment tree has billions of node, and each operation on it touches merely $O(\log 1000000000)$ 30 vertices, then almost all of the nodes will never be touched throughout the whole program; if they are not used, it is a waste to allocate memory for them. Because we know exactly what an untouched node looks like, and what information it holds (in this case, zero!).

Extending from the previous chapter, notice that when pointer allocation is used, we need not build the whole tree at the start. In this task, we allocate only one node at the start, the root node. This trick is named sparse segment tree or dynamic segment tree.

```

ii = 1;
root = ii++; /* root is at T[1] */
T[root].lc = T[root].rc = 0;

```

In query procedure, if `v` is 0, it means the node was never created. Hence we can act as if that section of the array is filled with zeros.

```

S query(int v, int tl, int tr, int l, int r) {
    if ( 0 == v || tr < l || r < tl) return 0;
    ...
}

```

This way, a query procedure will *never* make any new node; it need not do so. But update s might. And it can be implemented trivially, as follows.

```

int update(int v, int tl, int tr, int p, int val) {
    if ( v == 0 ) {
        v = ii++;
        T[v].lc = T[v].rc = 0;
    }
    ...
}

```

¹ If offline solutions are allowed. Standard Fenwick tree can solve this.

```

    }
    if (p <= mid) {
        T[v].lc = update(T[v].lc, tl, mid, p, val);
        T[v].rc = update(T[v].rc, mid + 1, tr, p, val);
    }
    pushup(v, tl, tr);
    return v;
}

```

Beware, each call to this function need to assign the return value back to the variable that was passed as the first argument.

Exercise 19. Implement a replacement for `std::set<int>` with sprase segment tree, where each operation takes $O(\log^2 \max)$

Exercise 20. SPOJ GSS3: maintain an array of integer $A[1..N]$ supporting these queries; (i) assign $A[p] = v$ (ii) given l, r , find the subarray of the subarray $A[l..r]$ with highest sum of elements. Forced $O(Q \log N)$.

3.10 Walking on Segment Tree

This is a similar but mightier procedure to subsection 2.6. Often you will want to, given a boolean function $check(x)$ and range $[l, r]$, find the lowest index $z \in [l, r]$ such that $check(query(l, z))$ is true—hereafter we call this function *findfirst*(). Similarly you may want to find the highest or rightmost index z —we call this *findlast*(). Since the two functions are of the same principle, we only discuss *findfirst* to avoid redundant details.

There is an obvious $O(\log^2 N)$ per query approach by binary searching and calling *query*. But they can be solved in $O(\log N)$ per query. First, consider an easier version *findfirstroot*(), which returns the lowest $z \in [1..N]$ where $check(query(1, z))$ is true. Notice that we cannot provide $[l, r]$ to the function here. An easy way to implement *findfirstroot* is to descend down the tree, eventually ending at a leaf. In the procedure, we keep a global ϕ value that is reset to e (identity) at the start; at node u we wish to go down the left child, as it holds lower indices. So we check if $\phi \circ T[lc(u)]$ satisfies *check*. If it does, then the answer z lies in the left child and we traverse there. Else, z lies in the right child and we can add $T[lc(u)]$ to ϕ . It is easy to see that this function recurses in $O(\log N)$ depth.

This *findfirstroot* implementation is easily extended to solve *findfirst*. Notice that any subarray can be decomposed into union of $O(\log N)$ disjoint intervals, each corresponding to a node in the segment tree. In fact, the standard *query* process does precisely that. Once we obtain those nodes, let it be $v_1, v_2, \dots, v_k$, iterate, adding up $T[v_i]$ along the way to find the first y such that $T[v_1] \circ T[v_2] \dots \circ T[v_y]$ satisfies *check*. If no such y exists, then no z exists. Else, set ϕ to be $T[v_1] \circ T[v_2] \dots \circ T[v_{y-1}]$ and perform *findfirstroot* on $T[v_y]$.

Exercise 21. Implement a replacement for `std::set<int>` with sprase segment tree, where each operation takes $O(\log \max)$

Exercise 22. Add in the result of the last exercise, the ability to fetch k^{th} maximum element in the set.

3.11 Range Update Range Query: Pushdown-Free

The problem below motivates this (and the next) topics.

Example 23. Maintain an array $A[1..N]$, initially zeroes, supporting these queries: $\text{update}(l, r, v)$ adds v to each of $A[l], A[l+1], \dots, A[r]$. $\text{getsum}(l, r)$ asks for the sum $A[l] + A[l+1] + \dots + A[r]$. Use $O(\log N)$ time per query.

This section presents the non-standard way to do it, which some refers to as “pushdown-free segment tree”. The next section presents the standard “lazy-propagation” technique.

Solution. Let us try extending 3.6. Recall that in that example, we maintain $\text{tag}[v]$ for each node v , storing sum of all the past updates which covers v ’s range. It turns out, that is enough information to perform range sum query. Consider the code below:

```
long long query(int v, int tl, int tr, int l, int r)
{
    if (tr < l || r < tl) return 0;
    if (l <= tl && tr <= r)
        return tag[v] * (long long)(r - l + 1);
    int overlap = min(tr, r) - max(tl, l) + 1;
    return overlap * (long long)tag[v]
        + query(lc(v), tl, mid, l, r)
        + query(rc(v), mid + 1, tr, l, r);
}
```

The main idea is, that the result to $\text{query}(l, r)$ is equal to the sum over every segment tree nodes, the product of its tag and the overlapping length between the tree range and query range. That is, $\sum_v \text{tag}[v] \cdot \text{overlap}([l, r], [tl_v, tr_v])$.

3.12 Range Update Range Query: Lazy Propagation.

Hitherto, we have used the invariant that each node if the segment tree always stores the correct aggregate data of its tree range; here we need to throw away that invariant. . Or rather, we modify it. In lazy propagation, we accept that nodes might store *outdated* information relative to their children, provided we have a way to fix it just-in-time.

Consider the Range Add, Range Sum problem again. If we perform an update adding v to the subarray $A[l..r]$, we find the $O(\log N)$ nodes in the cover set. For a node u fully covered by $[l, r]$, we add to the $\text{sum}[u]$ by $v \cdot \text{length}(u)$. But we wish not to recurse down further, for that breaks the $O(\log N)$ time complexity. Thus, instead of recursing down to u ’s children, we *accept* that its children (and their children, and so on...) will, for now, be ‘outdated’. We take note of this

fact in the array `lazy[u]`. This `lazy[u]` signifies that some updates (whose v sums to `lazy[u]`) has been applied to u but not its descendants.

When a later query or update operation needs to visit the children of u , we must ensure the children receive this pending update. This process is called *pushdown*. We apply the `lazy[u]` value to its two children's `sum[]`, then that `lazy[u]` can be resetted to zero, for the update has been 'propagated' to its children. Note that pushing down on u usually does not make all node in the subtree up-to-date, but its children will be.

Refer to the code below for clarity.

```
#define N 100000

#define lc(u) (2*(u))
#define rc(u) (2*(u)+1)
#define mid ((l + r)/2)

int sum[N * 4], lazy[N * 4];

void pushup(int v, int tl, int tr) {
    if (tl == tr) return;
    sum[v] = sum[lc(v)] + sum[rc(v)];
}

void pushdown(int v, int tl, int tr) {
    if (lazy[v] != 0) {
        int tm = (tl + tr) / 2;
        sum[lc(v)] += lazy[v] * (tm - tl + 1);
        sum[rc(v)] += lazy[v] * (tr - tm);
        lazy[lc(v)] += lazy[v];
        lazy[rc(v)] += lazy[v];
        lazy[v] = 0;
    }
}

void update(int v, int tl, int tr, int l, int r, int val) {
    if (tr < l || r < tl)
        return;
    if (l <= tl && tr <= r) {
        sum[v] += (long long)val * (tr - tl + 1);
        lazy[v] += val;
        return;
    }
    pushdown(v, tl, tr);
    update(lc(v), tl, mid, l, r, val);
```

```

        update(rc(v), mid + 1, tr, l, r, val);
        pushup(v, tl, tr);
    }

    long long query(int v, int tl, int tr, int l, int r) {
        if (tr < l || r < tl)
            return 0;
        if (l <= tl && tr <= r)
            return sum[v];
        pushdown(v, tl, tr);
        return query(lc(v), tl, mid, l, r) + query(rc(v), mid + 1, tr,
l, r);
    }

```

3.13 Generalization of Lazy Propagation

TODO

4 Linear Algebra

Vectors and Linear Combinations Here is presented the computational view-point of linear algebra. A *vector* is a list of reals, e.g., the vector $v = \langle 2, 3 \rangle$ has dimension 2. A vector $v = \langle v_1, v_2, \dots, v_k \rangle$ can be scaled by a real c , obtaining $cv = \langle cv_1, cv_2, \dots, cv_k \rangle$. Two vectors of the same dimension $v = \langle v_1, v_2, \dots, v_k \rangle$ and $w = \langle w_1, w_2, \dots, w_k \rangle$ may be added together, obtaining $v + w = \langle v_1 + w_1, v_2 + w_2, \dots, v_k + w_k \rangle$. One can also find their *dot product* $v \cdot w = v_1w_1 + v_2w_2 + \dots + v_kw_k$.

A vector $v = \langle v_1, v_2, \dots, v_k \rangle$ may also be written as $\begin{bmatrix} v_1 \\ v_2 \\ \vdots \\ v_k \end{bmatrix}$, or more compactly,

as the transpose of that: $[v_1, v_2, \dots, v_k]^T$.

Exercise 24. Find the dot product of $\langle 2, 3, 4 \rangle$ and $\langle 5, 1, 0 \rangle$.

Given a list of vectors $(v_1, v_2, \dots, v_m)$, their linear combinations are the values in form $c_1v_1 + c_2v_2 + \dots + c_mv_m$ where c_i are real-valued coefficients. For example, a linear combination of (v_1, v_2, v_3) above is $5v_1 + 2v_2 + 0v_3 = [12, 23, 15]^T$.

Exercise 25. Prove that zero is a linear combination of any list of vectors.

Exercise 26. Find the set of all linear combinations of the list $([2, 1])$.

Span

The set of all linear combinations of the list of vector $(\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_m)$ is said to be its *span*. The list spans its span.

Exercise 27. Proof that no list consisting of one vector can span a plane.

Exercise 28. Find $\text{span}(\langle 1, 2 \rangle, \langle 2, 1 \rangle)$.

Linear Independence

A list of vectors $(\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_m)$ is said to be *linearly independent* iff the only coefficients $c_1, c_2, \dots, c_m$ that forms a linear combination resulting in zero, is the trivial coefficients (that is $c_i = 0$ for all i). If the list is not linearly independent, it is said to be linearly dependent.

Exercise 29. Can a linearly dependent list of two vectors span a plane?

Exercise 30. Can a linearly dependent list of three vectors spans a three-dimensional space?

Exercise 31. What is the minimum number of vectors needed to span the N -dimensional space?

An equivalent definition of linearly independence: A list of vectors is linearly independent iff one of the vectors can be written as a nontrivial linear combination of the other vectors.

Exercise 32. Proof that the two definitions are equivalent.

Vector Spaces

A *vector space* is a set V of vectors together with addition of vectors, and multiplication of vectors by scalar; the following properties need to hold.

- Addition of vectors is commutative and associative.
- There is a zero vector, $\mathbf{0}$ (additive identity).
- There is a scalar one, 1 (multiplicative identity).
- Each vector has its additive inverse.

Note, that while previously we defined a vector to be a list of reals, a vector actually can be anything, so long as the vector space satisfies this property. Thus, **a vector is an element of a vector space.**

Basis

Matrices A *matrix* is a rectangular grid of numbers. Given a list of vectors $(v_1, v_2, \dots, v_m)$ where all of them has the same dimension n , one could write a matrix where the columns are the vectors.

If $v_1 = [2, 3, 1]^T$; $v_2 = [1, 4, 5]^T$, then the matrix whose columns are the two vectors in order is their column matrix, which is

$$\begin{bmatrix} \mathbf{v}_1 & \mathbf{v}_2 \end{bmatrix} = \begin{bmatrix} 2 & 1 \\ 3 & 4 \\ 1 & 5 \end{bmatrix}$$

The matrix above is said to be *3 by 2*; it has three rows and two columns. Its column vectors are $\mathbf{v}_1, \mathbf{v}_2$. The *column space* of a matrix is the span of all of its column vectors.

Matrix-Vector Product

Given a matrix A with m columns and a vector $\mathbf{x}$ of dimension m , their product $A\mathbf{x}$ is the linear combination of the matrix's column vectors where the coefficients are the components of the vector $\mathbf{x}$. That is, if the column vectors of A are $\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_m$, then,

$$A\mathbf{x} = x_1\mathbf{v}_1 + x_2\mathbf{v}_2 + \dots + x_m\mathbf{v}_m$$

For example, if $A = \begin{bmatrix} 2 & 3 \\ 4 & 5 \end{bmatrix}$ and $\mathbf{x} = [8, 7]^T$, then their product is $A\mathbf{x} = [37, 67]^T$.

Note, that for the product to be defined, the dimension of the vector must be the row length (that is, number of columns) of the matrix.

Matrix-Matrix Product

Given matrices A and B where the row length of A , denoted m , coincides with the column length of B , then, their product AB is defined as the column matrix of the vectors $A\mathbf{b}_1, A\mathbf{b}_2, \dots, A\mathbf{b}_k$ where $\mathbf{b}_i$ are the column vectors of B .

TODO

Guassain Elimination

TODO

5 Tree Flattening

6 DSU on Tree

7 Dynacon

8 References

- [1] Mircea Dima, and Rodica Ceterchi. Efficient Range Minimum Queries using Binary Indexed Trees. *Olympiads in Informatics*. 2015, 9 39–44. DOI 10.15388/loi.2015.0

associative 10
Binary Indexed Tree 4
column space 20
coordinate compression 3
discretization 3
dot product 19
Fenwick Tree 4
linearly independent 20
lowbit 4
magma 10
matrix 20
monoid 11
pushdown 18
Segment Tree 7
semigroup 10
span 19
spans 19
sparse segment tree 15
Twos Complement 4
update structure 13
vector 19
vector space 20
{column matrix}, 20
{dynamic segment tree}. 15